

Fundamentals of
PYTHON
DATA STRUCTURES

2ND EDITION

Kenneth A. Lambert





National Economics University
Faculty of Mathematical Economics

EP03.TITH1121 - DATA STRUCTURES AND ALGORITHMS: COLLECTIONS, ARRAYS, AND Linked Structures

- ❑ Instructor: Duc Minh Vu (MFE)
- ❑ Email: minhvd [at] neu.edu.vn



Learning Objectives

1. Recognize different categories of collections and the operations on them
2. Understand the difference between an abstract data type and the concrete data structures used to implement it
3. Perform basic operations on arrays, such as insertions and removals of items
4. Resize an array when it becomes too small or too large
5. Describe the space/time trade-offs for users of arrays
6. Perform basic operations, such as traversals, insertions, and removals, on linked structures
7. Explain the space/time trade-offs of arrays and linked structures in terms of the memory models that underlie these data structures



Reading

□ Theory

- [Array Data Structure – GeeksforGeeks](#)
- [Linked List Data Structure – GeeksforGeeks](#)

□ Practice:

- [Practice | GeeksforGeeks | A computer science portal for geeks](#)
- [Problems – LeetCode](#) (Array)
- [Problems – LeetCode](#) (Linked List)



Introduction

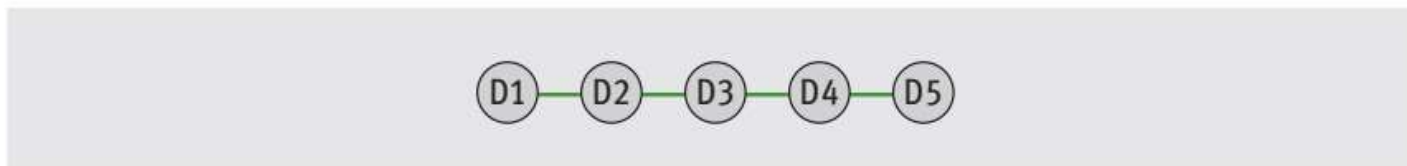
- ❑ Explore several frequently used classes called **collections**.
- ❑ Collections all have the same fundamental purpose—they help programmers organize data in programs effectively.
- ❑ Collections can be viewed from two perspectives. Users or clients of collections are concerned with what they do in various applications.
- ❑ Developers or implementers of collections are concerned with how they can best perform as general-purpose resources.
- ❑ Overview of different types of collections from the perspective of the users of those collections, and also introduce two data structures—arrays and linked structures—commonly used to implement collections



13.1 Overview of Collections

13.1.1 Linear Collections

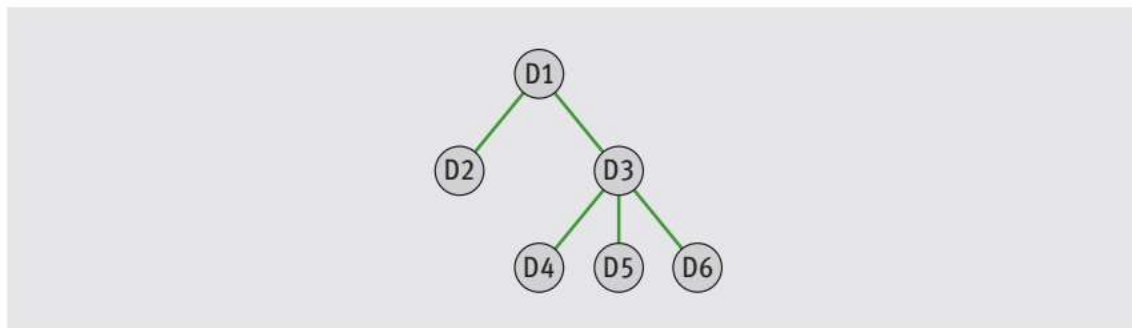
- ❑ The items in a **linear collection** are ordered by position.
- ❑ Each item except the first has a unique predecessor, and each item except the last has a unique successor.
- ❑ Everyday examples of linear collections are grocery lists, stacks of dinner plates, and a line of customers waiting at a bank.



[FIGURE 13.1] A linear collection

13.1.2 Hierarchical Collections

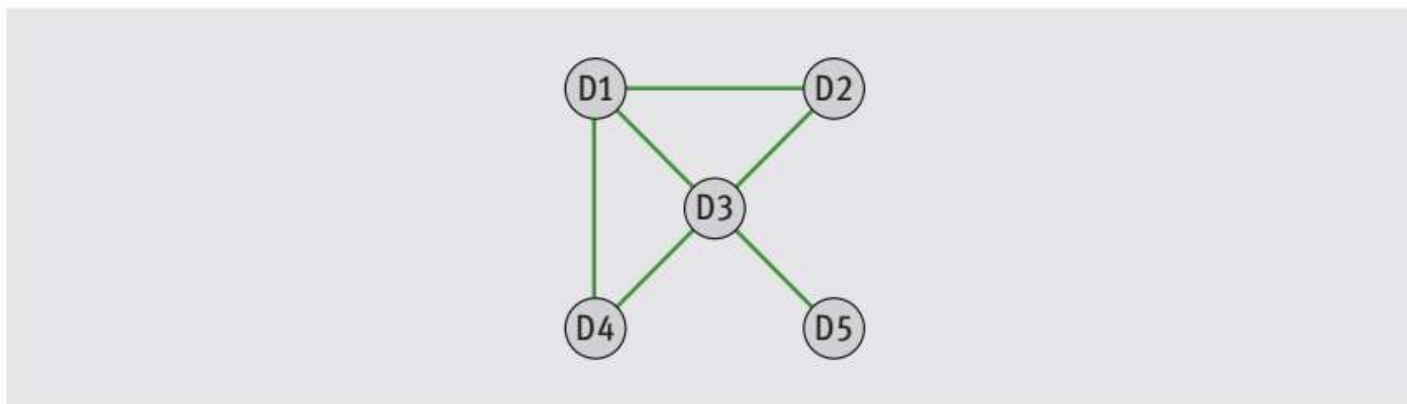
- ❑ Data items in **hierarchical collections** are ordered in a structure reminiscent of an upside-down tree.
- ❑ Each data item except the one at the top has just one predecessor, its **parent**, but potentially many successors, called its **children**.
- ❑ A file directory system, a company's organizational tree, and a book's table of contents are examples of hierarchical collections.



[FIGURE 13.2] A hierarchical collection

13.1.3 Graph Collections

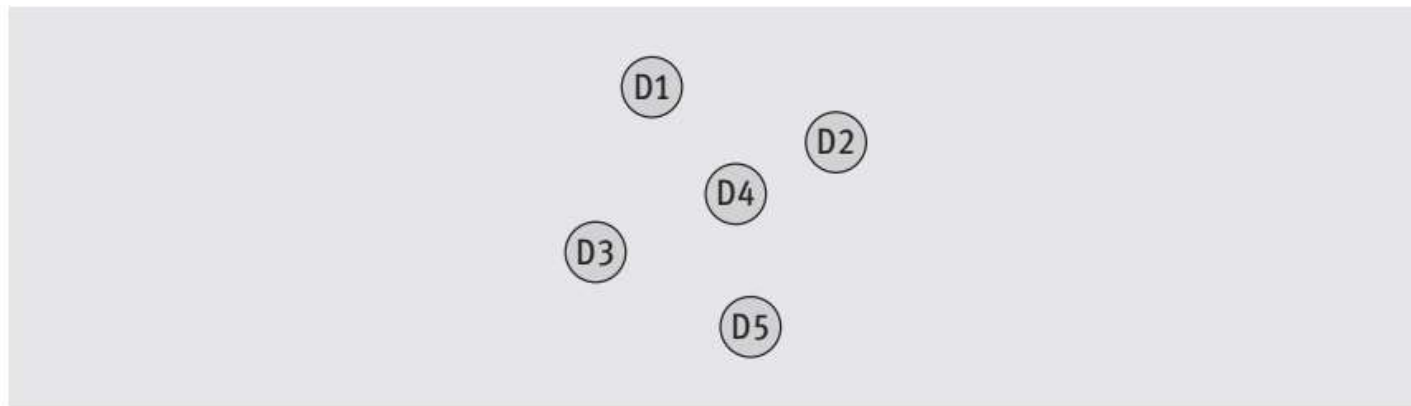
- ❑ A **graph collection**, also called a **graph**, is a collection in which each data item can have many predecessors and many successors.
- ❑ Examples of graphs are maps of airline routes between cities and electrical wiring diagrams for buildings.



[FIGURE 13.3] A graph collection

13.1.4 Unordered Collections

- items in an **unordered collection** are not in any particular order, and one cannot meaningfully speak of an item's predecessor or successor.
- A bag of marbles is an example of an unordered collection



[FIGURE 13.4] An unordered collection

13.1.5 Operations on Collections



CATEGORY OF OPERATION	DESCRIPTION
Search and retrieval	These operations search a collection for a given target item or for an item at a given position. If the item is found, either it or its position is returned. If the item is not found, a distinguishing value, such as None or -1 , is returned.
Removal	This operation deletes a given item or the item at a given position.

13.1.5 Operations on Collections



Insertion	This operation adds an item to a collection, usually at a particular position within the collection.
Replacement	This operation combines removal and insertion into a single operation.
Traversal	This operation visits each item in a collection. Depending on the type of collection, the order in which the items are visited can vary. During a traversal, items can be accessed or modified. Collections that can be traversed with Python's for loop are said to be iterable .
Test for equality	This operation tests two items to see if they are equal. If items can be tested for equality, then the collections containing them can also be tested for equality. To be equal, two collections must contain equal items at corresponding positions. For unordered collections, of course, the requirement of corresponding positions can be ignored. Some collections, such as strings, also can be tested for their position in a natural ordering using the comparisons less than and greater than.

13.1.5 Operations on Collections



Determine the size	This operation determines the size of a collection—the number of items it contains. Some collections also have a maximum capacity, or number of places available for storing items. An egg carton is a familiar example of a container with a maximum capacity.
Cloning	This operation creates a copy of an existing collection. The copy usually shares the same items as the original, a feat that is impossible in the real world. In the real world, a copy of a bag of marbles could not contain the same marbles as the original bag, given a marble's inability to be in two places at once. The rules of cyberspace are more flexible, however, and there are many situations in which we make these strange copies of collections. What we are copying is the structure of the collection, not the elements it contains. It is possible, however, and sometimes useful to produce a deep copy of a collection in which both the structure and the items are copied.

[TABLE 13.1] Categories of operations on collections



13.1.6 Abstraction and Abstract Data Types

- ❑ User's perspective: a collection is an abstraction, and for this reason, in computer science, collections are called **abstract data types (ADTs)**.
 - The user of an ADT is concerned only with learning its interface, or the set of operations that objects of that type recognize.
- ❑ Developers of collections are concerned with implementing a collection's behavior in the most efficient manner possible, with the goal of providing the best performance to users of the collections



13.1 Exercises

- 1 Give three examples of linear collections and hierarchical collections.
- 2 What is a graph collection? Give an example.
- 3 What is a traversal of a collection? Give an example.



13.2 Data Structures for Implementing Collections: Arrays



13.2.1 The Array Data Structure

- An array represents a sequence of items that can be accessed or replaced at given index positions.
 - The underlying data structure of a Python list is an array
 - The length or capacity of an array is fixed when it is created

USER'S ARRAY OPERATION	METHOD IN THE <code>Array</code> CLASS
<code>a = Array(10)</code>	<code>__init__(capacity, fillValue = None)</code>
<code>len(a)</code>	<code>__len__()</code>
<code>str(a)</code>	<code>__str__()</code>
<code>for item in a:</code>	<code>__iter__()</code>
<code>a[index]</code>	<code>__getitem__(index)</code>
<code>a[index] = newItem</code>	<code>__setitem__(index, newItem)</code>

[TABLE 13.2] Array operations and the methods of the `Array` class

```
class Array(object):
    """Represents an array."""

    def __init__(self, capacity, fillValue = None):
        """Capacity is the static size of the array.
        fillValue is placed at each position."""
        self._items = list()
        for count in xrange(capacity):
            self._items.append(fillValue)

    def __len__(self):
        """-> The capacity of the array."""
        return len(self._items)

    def __str__(self):
        """-> The string representation of the array."""
        return str(self._items)

    def __iter__(self):
        """Supports traversal with a for loop."""
        return iter(self._items)

    def __getitem__(self, index):
        """Subscript operator for access at index."""
        return self._items[index]

    def __setitem__(self, index, newItem):
        """Subscript operator for replacement at index."""
        self._items[index] = newItem
```



```
>>> a = Array(5)          # Create an array with 5 positions
>>> len(a)                # Show the number of positions
5
```

```
>>> print a               # Show the contents
[None, None, None, None, None]
>>> for i in xrange(len(a)): # Replace contents with 1..5
    a[i] = i + 1

>>> a[0]                  # Access the first item
1
>>> for item in a:         # Traverse the array to print all
    print item

1
2
3
4
5
```



13.3 Operations on Arrays

13.3.1 Increasing the Size of an Array

- ❑ When a new item is about to be inserted and the array's logical size equals its physical size, it is time to increase the size of the array.
- ❑ The resizing process consists of three steps:
 - 1 Create a new, larger array.
 - 2 Copy the data from the old array to the new array.
 - 3 Reset the old array variable to the new array object.

```
if logicalSize == len(a):  
    temp = Array(len(a) + 1)           # Create a new array  
    for i in xrange(logicalSize):      # Copy data from the old  
        temp[i] = a[i]                # array to the new array  
    a = temp                          # Reset the old array variable to the new array
```

13.3.2 Decreasing the Size of an Array

- ❑ When the logical size of an array shrinks, cells go to waste.
- ❑ When the number of these unused cells reaches or exceeds a certain threshold, say, three-fourths of the physical size of the array, it is time to decrease the physical size.
 - 1 Create a new, smaller array.
 - 2 Copy the data from the old array to the new array.
 - 3 Reset the old array variable to the new array object.

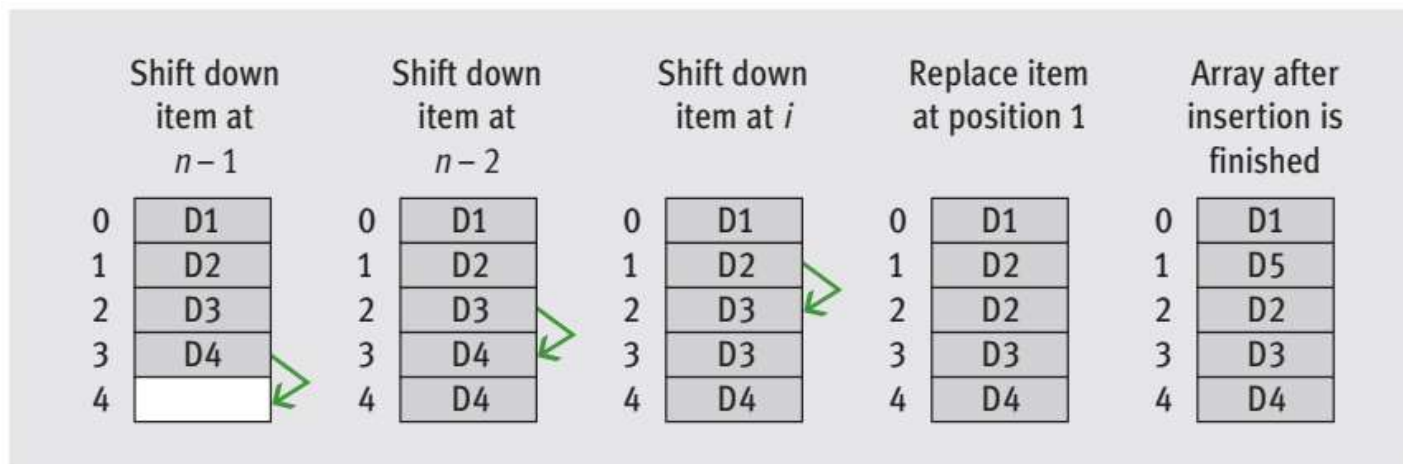
```
if logicalSize <= len(a) / 4 and len(a) > DEFAULT_CAPACITY:
    newSize = max(DEFAULT_CAPACITY, len(a) / 2)
    temp = Array(newSize)           # Create new array
    for i in xrange(logicalSize):   # Copy data from old array
        temp[i] = a[i]             # to new array
    a = temp                        # Reset old array variable to new array
```



13.3.3 Inserting an Item into an Array That Grows

□ In the case of an insertion, the programmer must do four things:

- 1 Check for available space before attempting an insertion and increase the physical size of the array, if necessary, as described earlier.
- 2 Shift the items from the logical end of the array to the target index position down by one. This process opens a hole for the new item at the target index.
- 3 Assign the new item to the target index position.
- 4 Increment the logical size by one



[FIGURE 13.7] Inserting an item into an array



```
# Increase physical size of array if necessary

# Shift items down by one position
for i in xrange(logicalSize, targetIndex, -1):
    a[i] = a[i - 1]

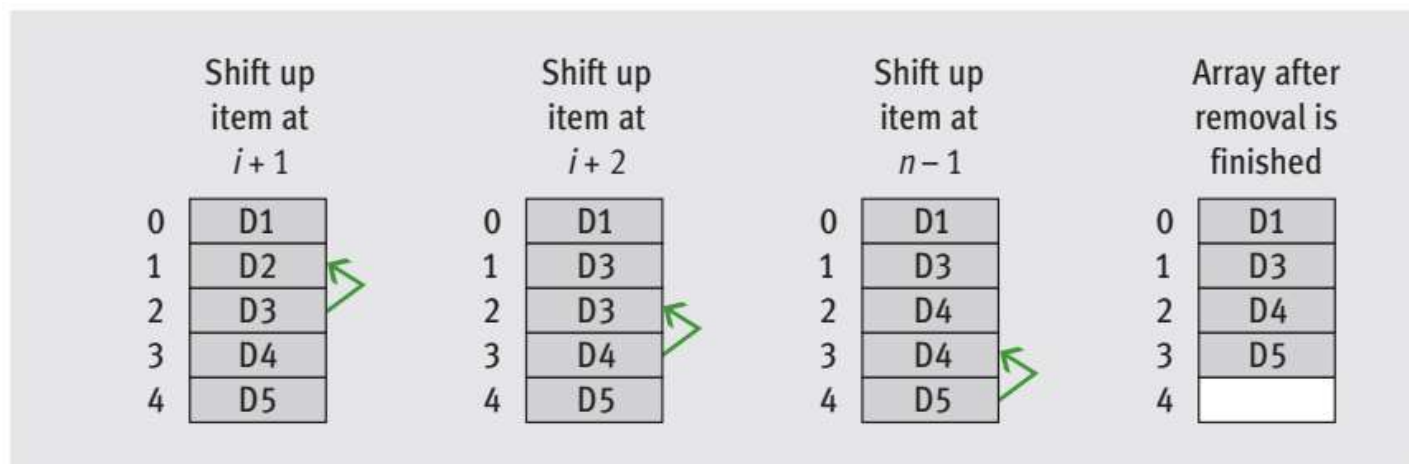
# Add new item and increment logical size
a[targetIndex] = newItem
logicalSize += 1
```



13.3.4 Removing an Item from an Array

□ In the case of a deletion, the programmer must do four things:

- 1 Shift the items from the target index position to the logical end of the array up by one. This process closes the hole left by the removed item at the target index.
- 2 Decrement the logical size by one.
- 3 Check for wasted space and decrease the physical size of the array, if necessary.



[FIGURE 13.8] Removing an item from an array



```
# Shift items up by one position
for i in xrange(targetIndex, logicalSize - 1):
    a[i] = a[i + 1]

# Decrement logical size
logicalSize -= 1

# Decrease size of array if necessary
```

13.3.5 Complexity Trade-Off: Time, Space, and Arrays



OPERATION	RUNNING TIME
Access at i th position	$O(1)$ (best and worst case)
Replacement at i th position	$O(1)$ (best and worst case)
Insert at logical end	$O(1)$ (average case)
Remove from logical end	$O(1)$ (average case)
Insert at i th position	$O(n)$ (average case)
Remove from i th position	$O(n)$ (average case)
Increase the capacity	$O(n)$ (best and worst case)
Decrease the capacity	$O(n)$ (best and worst case)

[TABLE 13.3] The running times of array operations



13.3

Exercises

- 1 Explain why some items in an array might have to be shifted when a given item is inserted or removed.
- 2 When the programmer shifts array items during an insertion, which item is moved first, the one at the insertion point or the last item, and why?
- 3 State the run-time complexity for inserting an item when the insertion point is the logical size of the array.
- 4 An array currently contains 14 items and its load factor is 0.70. What is its physical capacity?



13.4 Two-Dimensional Arrays (Grids)



13.4.1 Processing a Grid

□ A grid must recognize two methods that return the number of rows and the number of columns.

➤ These methods are named **getHeight** and **getWidth**, respectively

```
sum = 0
for row in xrange(table.getHeight()):      # Go through rows
    for column in xrange(table.getWidth()): # Go through columns
        sum += table[row][column]
```



13.4.2 Creating and Initializing a Grid

- Assume that there exists a **Grid** class for two-dimensional arrays.
- To create a **Grid** object, we run the **Grid** constructor with three arguments: its height, its width, and an initial fill value.

```
>>> table = Grid(4, 5, 0)
>>> print table
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

```
for row in xrange(table.getHeight()):          # Go through rows
    for column in xrange(table.getWidth()):    # Go through columns
        table[row][column] = int(str(row) + str(column))
```



13.4.3 Defining a Grid Class

- A grid is conveniently represented using an array of arrays.
 - The length of the top-level array equals the number of rows in the grid.
 - Each cell in the top-level array is also an array.
 - The length of this array is the number of columns in the grid, and this array contains the data in a given row.

```
"""
File: grid.py
"""

from arrays import Array

class Grid(object):
    """Represents a two-dimensional array."""

    def __init__(self, rows, columns, fillValue = None):
        self._data = Array(rows)
        for row in xrange(rows):
            self._data[row] = Array(columns, fillValue)

    def getHeight(self):
        """Returns the number of rows."""
        return len(self._data)

    def getWidth(self):
        """Returns the number of columns."""
        return len(self._data[0])
```

continued



```
def __getitem__(self, index):  
    """Supports two-dimensional indexing  
    with [row][column]."""  
    return self._data[index]  
  
def __str__(self):  
    """Returns a string representation of the grid."""  
    result = ""  
    for row in xrange(self.getHeight()):  
        for col in xrange(self.getWidth()):  
            result += str(self._data[row][col]) + " "  
        result += "\n"  
    return result
```



13.4.4 Ragged Grids and Multidimensional Arrays

- ❑ It is also possible to create ragged grids and grids of more than two dimensions.
- ❑ In a ragged grid, there are a fixed number of rows, but the number of columns of data in each row can vary.
- ❑ Dimensions can also be added to the definition of a grid when necessary
 - 3D array



13.4

Exercises

- 1 What are two-dimensional arrays or grids?
- 2 Describe an application in which a two-dimensional array might be used.
- 3 Write a code segment that searches a **Grid** object for a negative integer. The loop should terminate at the first instance of a negative integer in the grid, and the variables **row** and **column** should be set to the position of that integer. Otherwise, the variables **row** and **column** should equal the number of rows and columns in the grid.



- 4 Describe the contents of the grid after the following code segment is run:

```
matrix = Grid(3, 3)

for row in xrange(matrix.getHeight()):
    for column in xrange(matrix.getWidth()):
        matrix[row][column] = row * column
```

- 5 Write a code segment that creates a ragged grid whose rows contain positions for 3, 6, and 9 items, respectively.
- 6 Suggest a strategy for implementing a three-dimensional array class that uses the **Grid** class as a data structure.
- 7 Write a code segment that initializes each cell in a three-dimensional array with an integer that represents its three index positions. Thus, if a position is (depth, row, column), the integer datum at position (2, 3, 3) is 233.



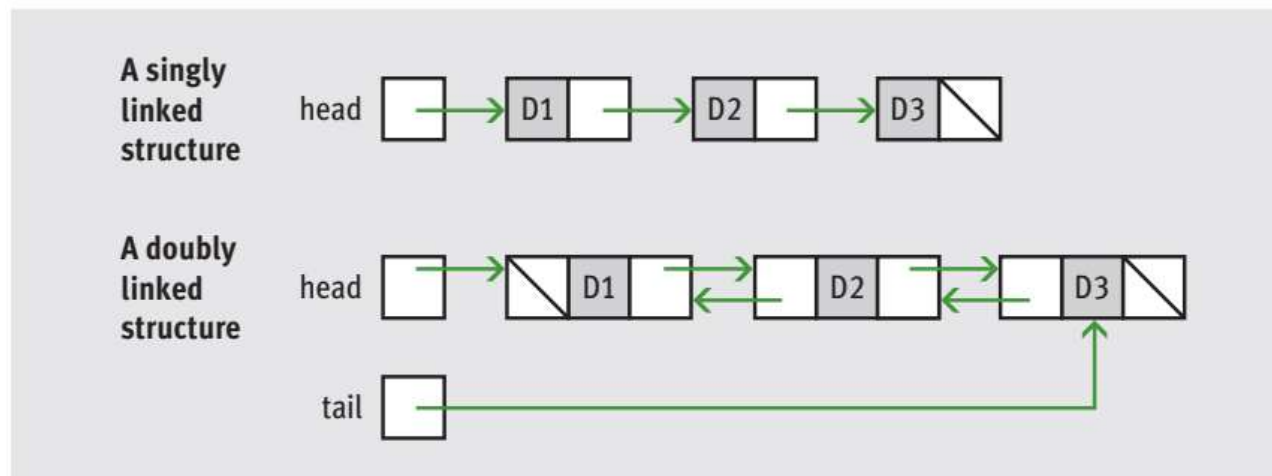
- 8 Write a code segment that displays the items in a three-dimensional array. Each line of data should represent all of the items at a given row and column, stretching back from the first depth position to the last one. The traversal should start at the first row, column, and depth positions and move through depths, columns, and rows.



13.5 Linked Structures

13.5.1 Singly Linked Structures and Doubly Linked Structures

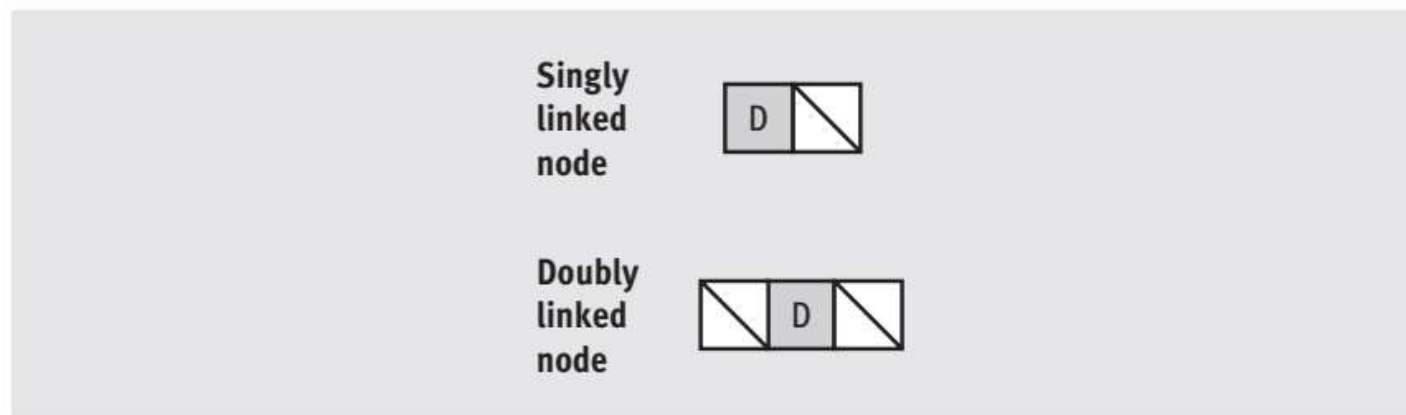
- ❑ A linked structure consists of items that are linked to other items.
- ❑ The two simplest linked structures are the **singly linked structure** and the **doubly linked structure**



[FIGURE 13.10] Two types of linked structures

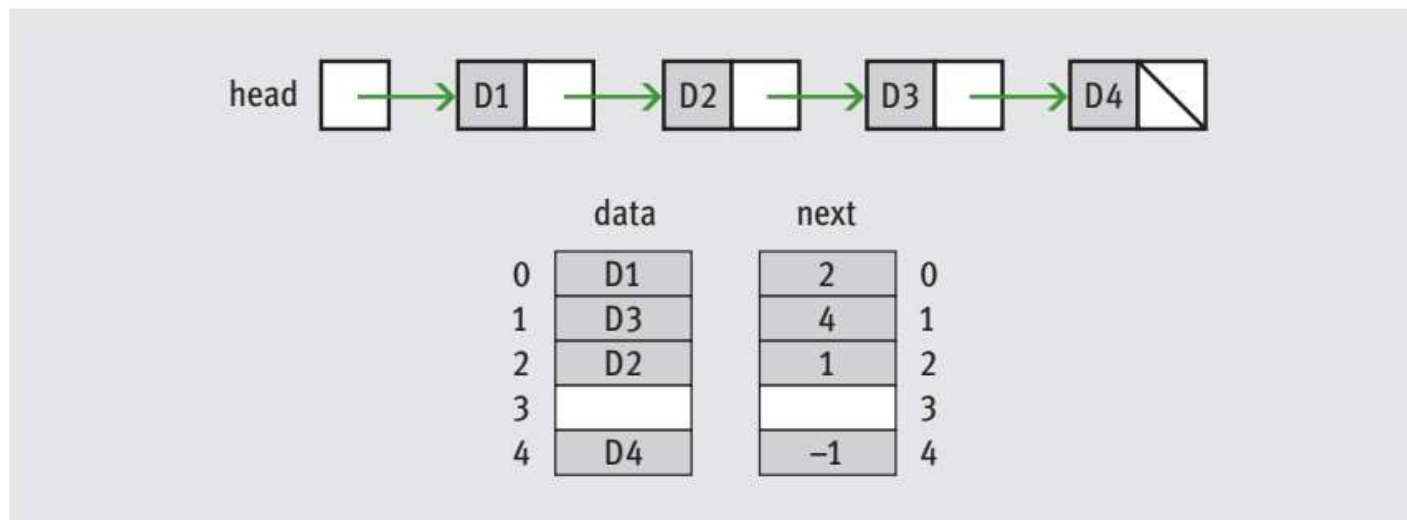
13.5.2 Noncontiguous Memory and Nodes

□ **noncontiguous memory:** the cell for a given item in a linked structure can be found anywhere in memory as long as the computer can follow a link to its address or location.



[FIGURE 13.11] Two types of nodes with empty links

Non-continuous memory



[FIGURE 13.12] An array representation of a linked structure



13.5.3 Defining a Singly Linked Node Class

```
class Node(object):  
  
    def __init__(self, data, next = None):  
        """Instantiates a Node with default next of None"""  
        self.data = data  
        self.next = next
```

13.5.4 Using the Singly Linked Node Class

```
# Just an empty link
```

```
node1 = None
```

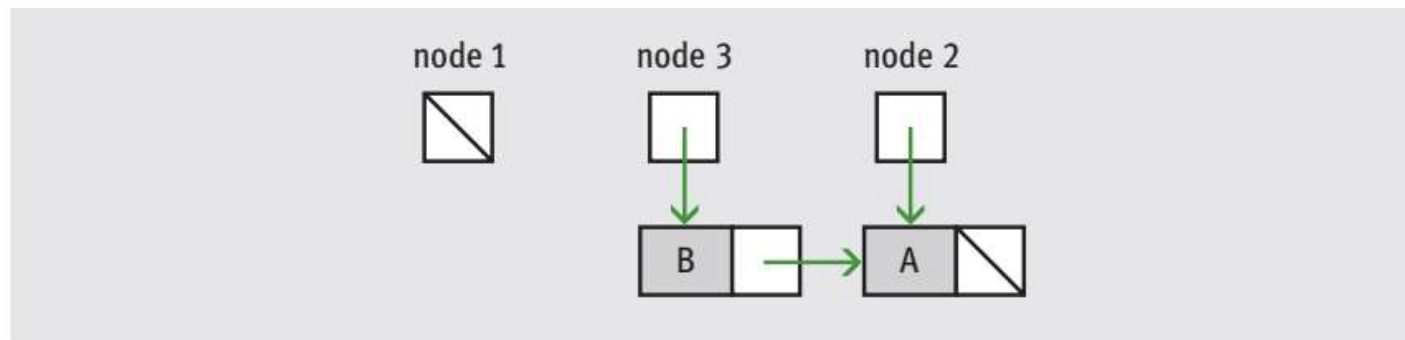
```
# A node containing data and an empty link
```

```
node2 = Node("A", None)
```

```
# A node containing data and a link to node2
```

```
node3 = Node("B", node2)
```

- **node1** points to no node object yet (is **None**).
- **node2** and **node3** point to objects that are linked together.
- **node2** points to an object whose next pointer is **None**.



[FIGURE 13.13] Three external links



```
"""
File: testnode.py

Tests the Node class.
"""

from node import Node

head = None

# Add five nodes to the beginning of the linked structure
for count in xrange(1, 6):
    head = Node(count, head)

# Print the contents of the structure
while head != None:
    print head.data
    head = head.next
```




13.5 Exercises

- 1 Using box and pointer notation, draw a picture of the nodes created by the first loop in the tester program.
- 2 What happens when a programmer attempts to access a node's data fields when the node variable refers to **None**? How does one guard against it?
- 3 Write a code segment that transfers items from a full array to a singly linked structure. The operation should preserve the ordering of the items.

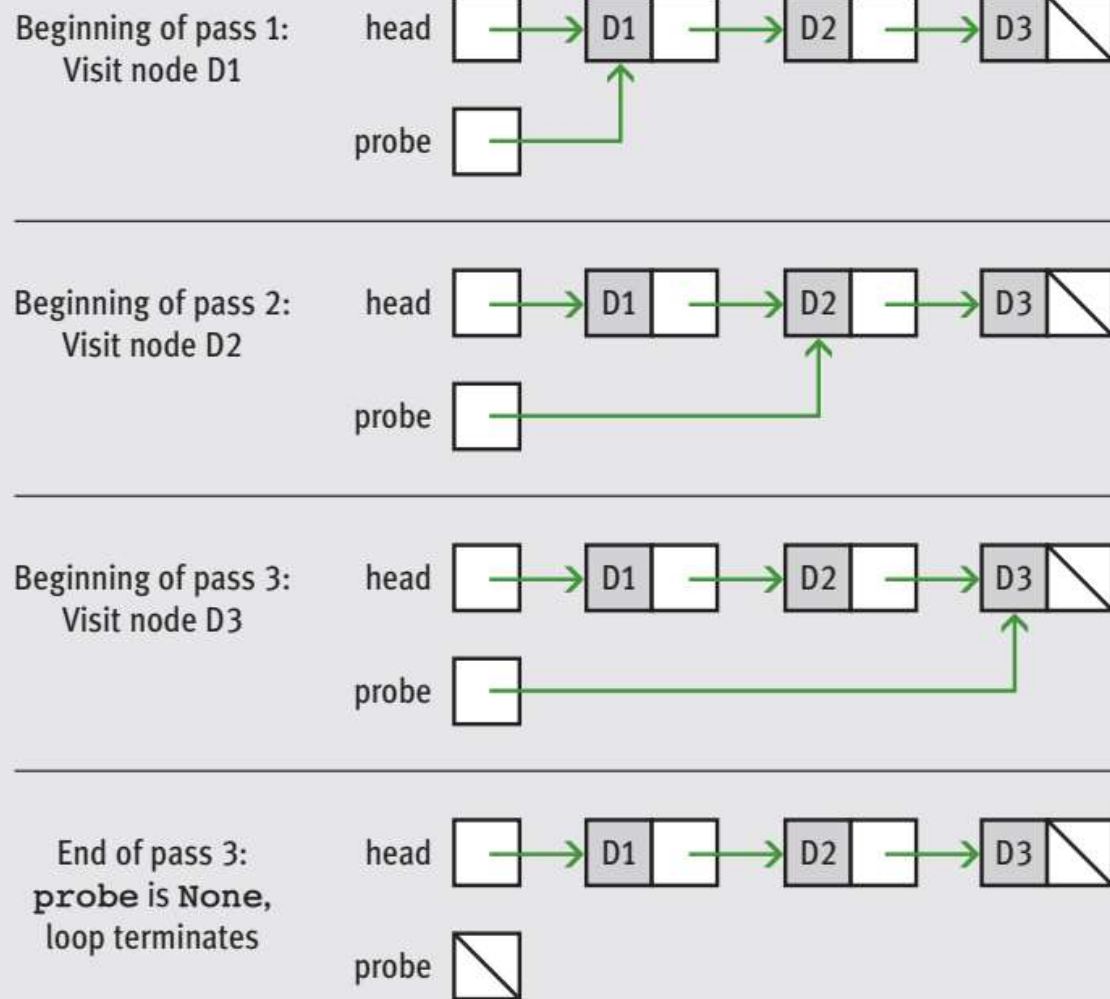


13.6 Operations on Singly Linked Structures



13.6.1 Traversal

```
probe = head
while probe != None:
    <use or modify probe.data>
    probe = probe.next
```



[FIGURE 13.14] Traversing a linked structure



13.6.2 Searching

□ Search for a given item

```
probe = head
while probe != None and targetItem != probe.data:
    probe = probe.next
if probe != None:
    <targetItem has been found>
else:
    <targetItem is not in the linked structure>
```



13.6.2 Searching

□ Accessing the i th item:

```
# Assumes 0 <= index < n
probe = head
while index > 0:
    probe = probe.next
    index -= 1
return probe.data
```



13.6.3 Replacement

```
probe = head
while probe != None and targetItem != probe.data:
    probe = probe.next;
if probe != None:
    probe.data = newItem
    return True
else:
    return False
```



```
# Assumes 0 <= index < n
probe = head
while index > 0:
    probe = probe.next
    index -= 1
probe.data = newItem
```


13.6.4 Inserting at the Beginning

Two cases:

➤ Empty list $\Leftrightarrow$ head is none

➤ Non-empty list $\Leftrightarrow$ head is not none.

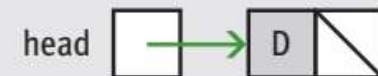
Inserting data at the beginning of a linked structure uses constant time and memory.

First case: head is None

Initial state of head

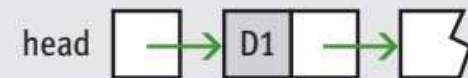


`head = Node(newItem, head)`



Second case: head is not None

Initial state of head



`head = Node(newItem, head)`



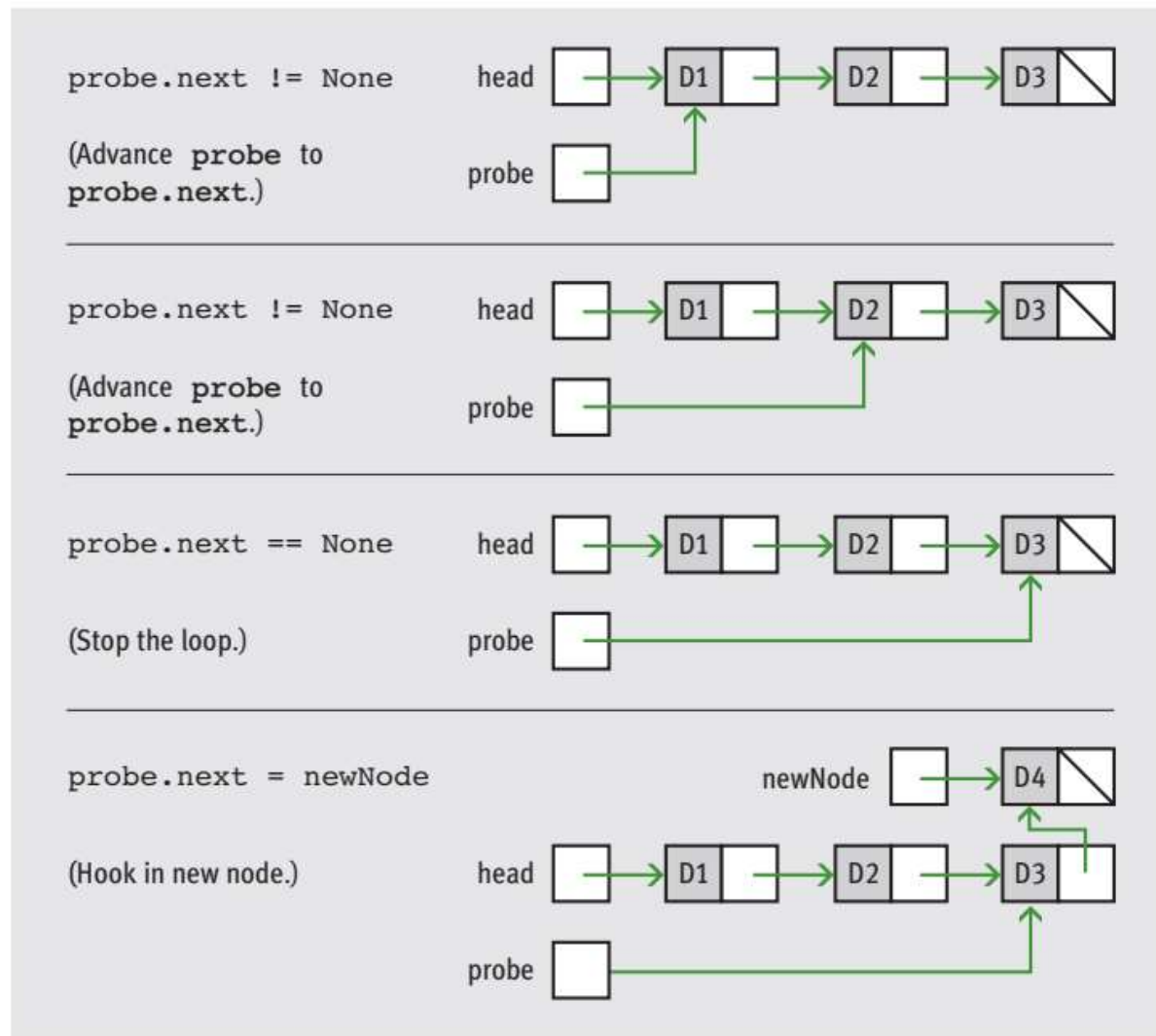
[FIGURE 13.15] The two cases of inserting an item at the beginning of a linked structure

13.6.5 Inserting at the End

□ Two cases:

- 1 The **head** pointer is **None**, so the head pointer is set to the new node.
- 2 The **head** pointer is not **None**, so the code searches for the last node and aims its next pointer at the new node.

```
newNode = Node(newItem)
if head is None:
    head = newNode
else:
    probe = head
    while probe.next != None:
        probe = probe.next
    probe.next = newNode
```

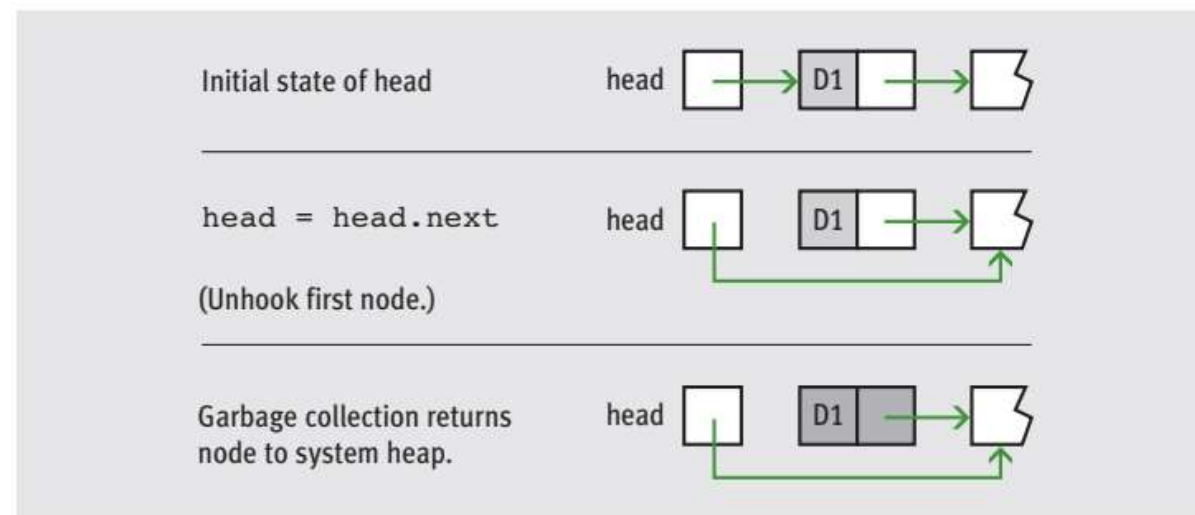


[FIGURE 13.16] Inserting an item at the end of a linked structure
Data Structures and Algorithms in Python

13.6.6 Removing at the Beginning

- Update the pointer of the head.
- the operation uses constant time and memory, unlike the same operation for arrays.

```
# Assumes at least one node in the structure  
removedItem = head.data  
head = head.next  
return removedItem
```



[FIGURE 13.17] Removing an item at the beginning of a linked structure

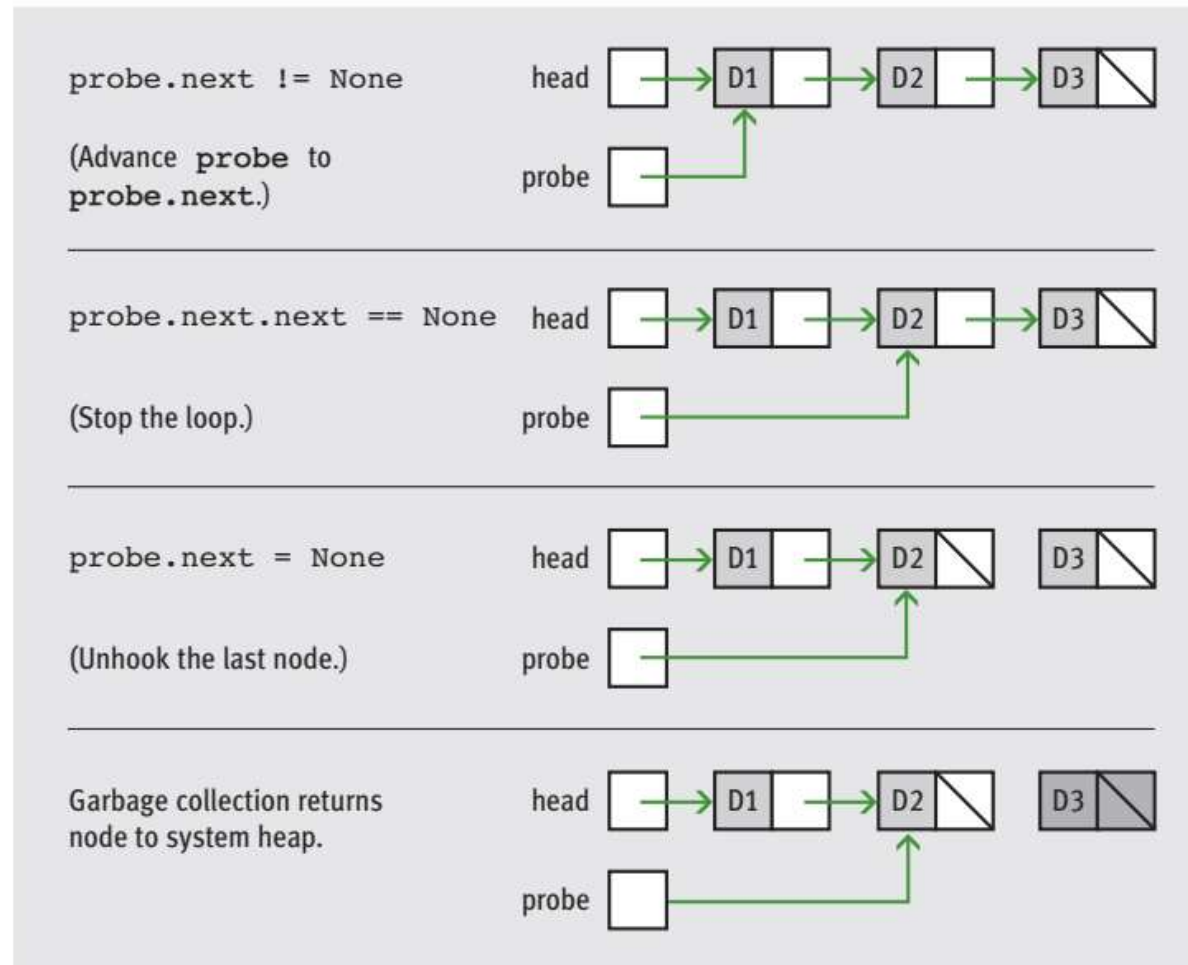
13.6.7 Removing at the End

❑ Two cases to consider:

- 1 There is just one node. The **head** pointer is set to **None**.
- 2 There is a node before the last node. The code searches for this second-to-last node and sets its next pointer to **None**.

❑ The operation is linear in time and constant in memory.

```
# Assumes at least one node in structure
removedItem = head.data
if head.next is None:
    head = None
else:
    probe = head
    while probe.next.next != None:
        probe = probe.next
    removedItem = probe.next.data
    probe.next = None
return removedItem
```



[FIGURE 13.18] Removing an item at the end of a linked structure

13.6.8 Inserting at Any Position

□ Insertion of an item at the i -th position - two cases to consider:

- 1 Node's next pointer is **None**. This means that $i \geq n$, so the new item should be placed at the end of the linked structure.
- 2 Node's next pointer is not **None**. That means that $0 < i < n$, so the new item must be placed between the node at position $i - 1$ and the node at position i .

□ $O(n)$ time and $O(1)$ memory

```
if head is None or index <= 0:
    head = Node(newItem, head)
else:
    # Search for node at position index - 1 or the last position
    probe = head
    while index > 1 and probe.next != None:
        probe = probe.next;
        index -= 1
    # Insert new node after node at position index - 1
    # or last position
    probe.next = Node(newItem, probe.next)
```

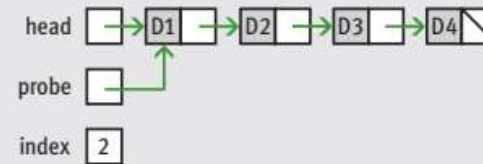


13.6.9 Removing at Any Position

- The removal of the i th item from a linked structure has three cases:
 - 1 $i \leq 0$. We use the code to remove the first item.
 - 2 $0 < i < n$. We search for the node at position $i - 1$, as in insertion, and remove the following node.
 - 3 $i \geq n$. We remove the last node.



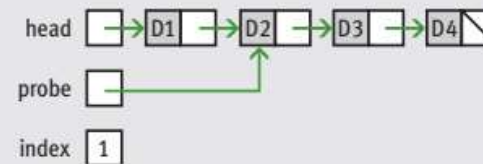
`index > 1 and
probe.next.next != None`



(Advance **probe** to
probe.next and
decrement **index**.)

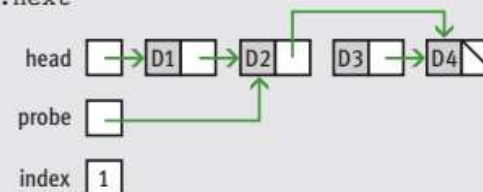
`index == 1`

(Stop the loop.)

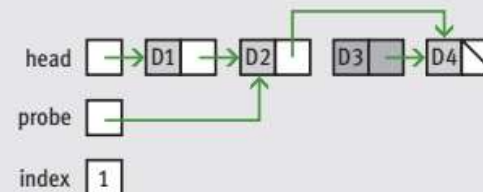


`probe.next = probe.next.next`

(Unhook the node.)



Garbage collection returns
node to system heap.



[FIGURE 13.20] Removing an item between two items in a linked structure



OPERATION	RUNNING TIME
Access at i th position	$O(n)$ (average case)
Replacement at i th position	$O(n)$ (average case)
Insert at beginning	$O(1)$ (best and worst case)
Remove from beginning	$O(1)$ (best and worst case)
Insert at i th position	$O(n)$ (average case)
Remove from i th position	$O(n)$ (average case)

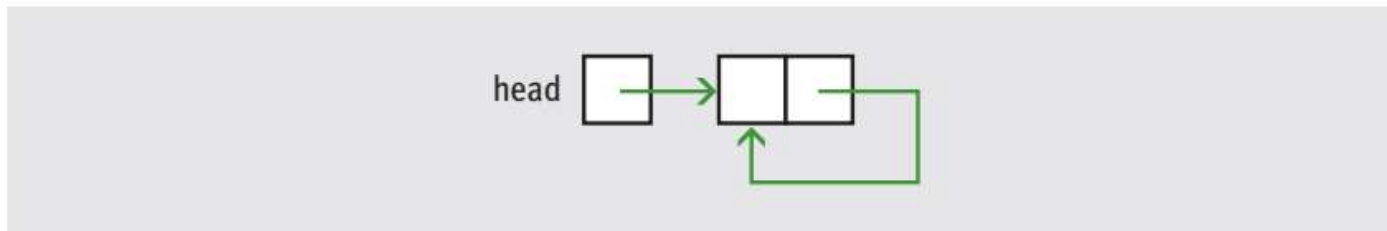
[TABLE 13.4] The running times of operations on singly linked structures



13.7 Variations on a Link

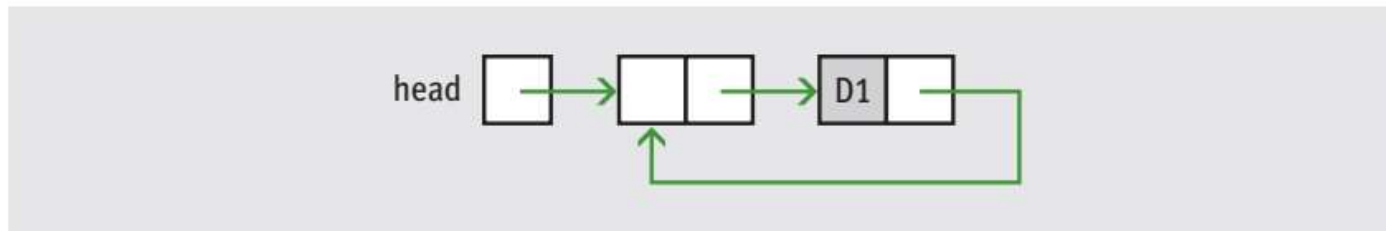
13.7.1 A Circular Linked Structure with a Dummy Header Node

- ❑ **Circular linked structure with a dummy header node:** a circular linked structure contains a link from the last node back to the first node in the structure. There is always at least one node in this implementation.
- ❑ This node, the dummy header node, contains no data, but serves as a marker for the beginning and the end of the linked structure.



[FIGURE 13.21] An empty circular linked structure with a dummy header node

13.7.1 A Circular Linked Structure with a Dummy Header Node



[FIGURE 13.22] A circular linked structure after inserting the first node

```
# Search for node at position index - 1 or the last position
probe = head
while index > 0 and probe.next != head:
    probe = probe.next
    index -= 1
# Insert new node after node at position index - 1 or
# last position
probe.next = Node(newItem, probe.next)
```

13.7.2 Doubly Linked Structures

It allows the user to do the following:

- 1 Move left, to the previous node, from a given node.
- 2 Move immediately to the last node.

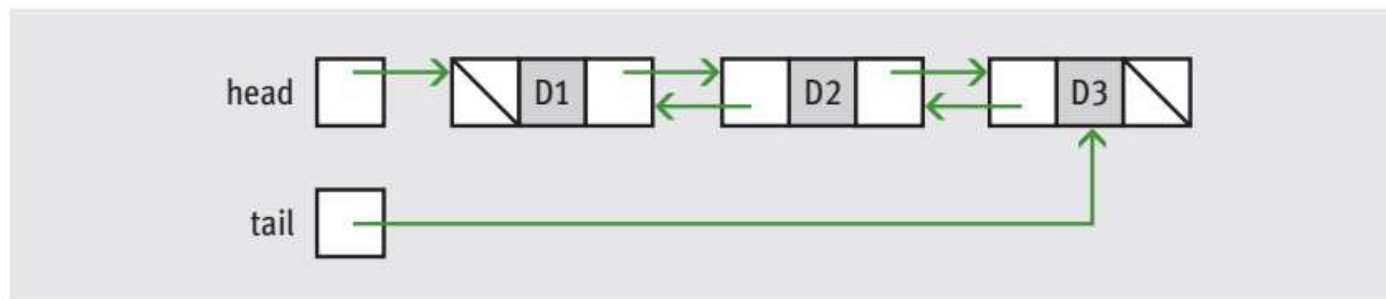


FIGURE 13.23 A doubly linked structure with three nodes



- The Python implementation of a node class for doubly linked structures extends the **Node** class discussed earlier by adding a field for the **previous** pointer.

```
class Node(object):  
  
    def __init__(self, data, next = None):  
        """Instantiates a Node with default next of None"""  
        self.data = data  
        self.next = next  
  
class TwoWayNode(Node):  
  
    def __init__(self, data, previous = None, next = None):  
        """Instantiates a TwoWayNode."""  
        Node.__init__(self, data, next)  
        self.previous = previous
```



□ creates a doubly linked structure by adding items to the end.

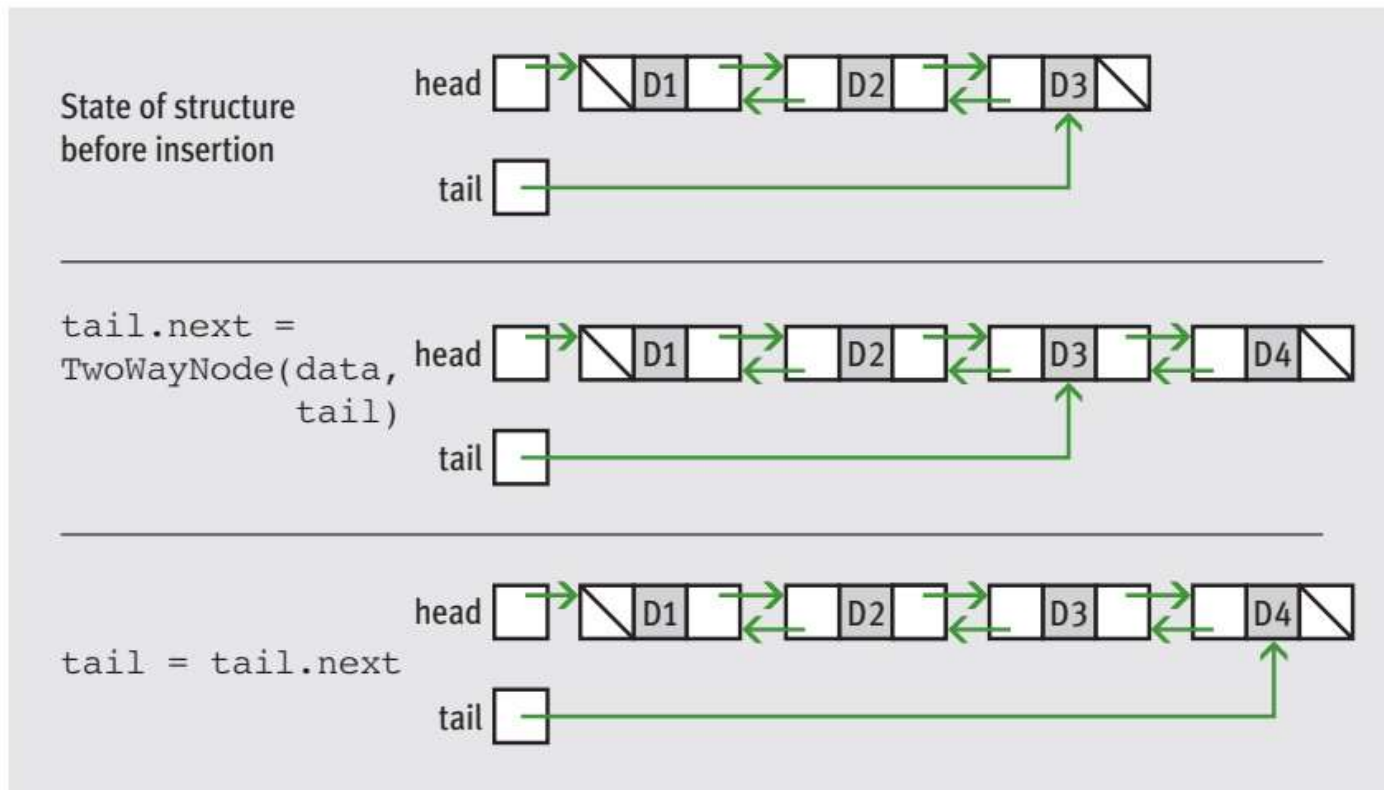
```
"""
File: testtwowaynode.py
Tests the TwoWayNode class.
"""

from node import TwoWayNode

# Create a doubly linked structure with one node
head = TwoWayNode(1)
tail = head

# Add four nodes to the end of the doubly linked structure
for data in xrange(2, 6):
    tail.next = TwoWayNode(data, tail)
    tail = tail.next

# Print the contents of the linked structure in reverse order
probe = tail
while probe != None:
    print probe.data
    probe = probe.previous
```

[FIGURE 13.24] Inserting an item at the end of a doubly linked structure



13.7

Exercises

- 1 What advantage does a circular linked structure with a dummy header node give the programmer?
- 2 Describe one benefit and one cost of a doubly linked structure, as compared to a singly linked structure.

REVIEW QUESTIONS

- 1 Examples of linear collections are
 - a sets and trees
 - b lists and stacks
- 2 Examples of unordered collections are
 - a queues and lists
 - b sets and dictionaries
- 3 A hierarchical collection can represent a
 - a line of customers at a bank
 - b a file directory system
- 4 A graph collection can represent a
 - a set of numbers
 - b map of flight paths between cities
- 5 Arrays and linked structures are examples of
 - a abstract data types (ADTs)
 - b data structures

- 6 An array's length
 - a is fixed in size after it is created
 - b can be increased or decreased after it is created
- 7 Random access supports
 - a constant time access to data
 - b linear time access to data
- 8 Data in a singly linked structure are contained in
 - a cells
 - b nodes
- 9 Most operations on singly linked structures run in
 - a constant time
 - b linear time
- 10 It requires constant time to remove the first item from a(n)
 - a array
 - b singly linked structure

PROJECTS

In the first six projects, you modify the **Array** class defined in this chapter to make it behave more like Python's **list** class. For each solution, include code that tests your modifications to the **Array** class.

- 1 Add an instance variable **__logicalSize** to the **Array** class. This variable is initially 0, and will track the number of items currently available to users of the array. Then add the method **size()** to the **Array** class. This method should return the array's logical size. The method **__len__** should still return the array's capacity or physical size.
- 2 Add preconditions to the methods **__getitem__** and **__setitem__** of the **Array** class. The precondition of each method is $0 \leq \text{index} < \text{size}()$. Be sure to raise an exception if the precondition is not satisfied.



- 3 Add the methods **grow** and **shrink** to the **Array** class. These methods should use the strategies discussed in this chapter to increase or decrease the length of the list contained in the array. Make sure that the physical size of the array does not shrink below the user-specified capacity and that the array's cells use the fill value when the array's size is increased.
- 4 Add the methods **insert** and **remove** to the **Array** class. These methods should use the strategies discussed in this chapter, including adjusting the length of the array, if necessary. The **insert** method expects a position and an item as arguments and inserts the item at the given position. If the position is greater than or equal to the array's logical size, the method inserts the item after the last item currently available in the array. The **remove** method expects a position as an argument and removes and returns the item at that position. The **remove** method's precondition is $0 \leq \text{index} < \text{size}()$. The **remove** method should also reset the vacated array cell to the fill value.



- 5 Add the method `__eq__` to the **Array** class. Python runs this method when an **Array** object appears as the left operand of the `==` operator. The method returns **True** if its argument is also an **Array**, it has the same logical size as the left operand, and the pair of items at each *logical* position in the two arrays are equal. Otherwise, the method returns **False**.
- 6 Jill tells Jack that he should now remove the current implementation of the `__iter__` method from the **Array** class, if it's really behaving like a list. Explain why this is a good suggestion. Also explain how the `__str__` method should be modified at this point.



- 7 A **Matrix** class can be used to perform some of the operations found in linear algebra, such as matrix arithmetic. Develop a **Matrix** class that uses the built-in operators for arithmetic in a manner similar to the **Rational** number class discussed in Chapter 8. The **Matrix** class should extend the **Grid** class.

The next four projects ask you to define some functions for manipulating linked structures. You should use the **Node** and **TwoWayNode** classes, as defined in this chapter. Create a tester module that contains your function definitions and your code for testing them.

- 8 Define a function **length** (*not* **len**) that expects a singly linked structure as an argument. The function returns the number of items in the structure.
- 9 Define a function named **insert** that inserts an item into a singly linked structure at a given position. The function expects three arguments: the item, the position, and the linked structure (the latter may be empty).

The function returns the modified linked structure. If the position is greater than or equal to the structure's length, the function inserts the item at its end. An example call of the function, where **head** is a variable that either is an empty link or refers to the first node of a structure, is **head = insert(1, data, head)**.

- 10 Define a function named **remove** that removes the item at a given position from a singly linked structure. This function expects a position as a first argument, with the precondition **$0 \leq \text{position} < \text{length of structure}$** . Its second argument is the linked structure, which, of course, cannot be empty. The function returns a *tuple* containing the modified linked structure and the item that was removed. An example call is **(head, item) = remove(1, head)**.
- 11 Define a function **makeTwoWay** that expects a singly linked structure as its argument. The function builds and returns a doubly linked structure that contains the items in the singly linked structure. (*Note: The function should not alter the argument's structure.*)